

JAVA INTERVIEW PREPARATION

CHAPTER 48

DATABASE INDEXES AND
QUERY PLANS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Database Indexes and Query Plans: Designing Predictable SQL Performance

Senior / Lead Java Developer Reading Chapter

An index is a maintained access structure that trades storage and write cost for faster retrieval. Senior database performance work begins with a real query shape, data distribution, and execution plan. Adding indexes from column names alone creates redundant structures, slower writes, and false confidence.

How a B-Tree Index Helps

A B-tree keeps ordered keys with row references. It supports equality, ranges, ordered scans, and prefix matching on a composite key.

```
root
-> branch [customer 1..500]
    -> leaf [(customer_id, created_at) -> row location]
```

The database still pays to traverse the tree and fetch table pages. When a predicate returns much of the table, a sequential scan can be cheaper. Big-O notation alone ignores page cache, random I/O, correlation, row width, and parallelism.

Composite Index Order

Design a composite index from equality predicates, range predicates, ordering, and projection.

```
select id, total
from orders
where tenant_id = :tenant
    and status = 'OPEN'
    and created_at < :cursor
order by created_at desc, id desc
limit 50;
```

A candidate index is `(tenant_id, status, created_at desc, id desc)`. The leading equality columns narrow the range; the remaining keys support deterministic ordering and keyset pagination.

An index on `(a,b)` commonly supports `a` and `a+b` access, but not an unconstrained `b` lookup efficiently. Do not memorize one universal "most selective first" rule. Match the full workload, ordering, and engine capabilities.

Selectivity and Statistics

Selectivity describes how strongly a predicate reduces rows. A boolean status alone may be a poor index when half the table matches. Combined with tenant and time, it may be excellent.

The optimizer estimates row counts from statistics. Skew, correlated columns, stale statistics, or parameter values can make estimates wrong. One tenant may own 60 percent of rows while most own hundreds; a plan good for the median tenant can be disastrous for the largest.

Compare estimated rows with actual rows in `EXPLAIN ANALYZE`. Large divergence points to statistics, correlation, parameter sensitivity, or a predicate the optimizer cannot model well.

Reading an Execution Plan

Read a plan from data-producing nodes upward. For each node inspect estimated versus actual rows, loops, time, buffers, sort or hash memory, and filters that discarded rows.

```
Limit
-> Index Scan using ix_orders_tenant_created
   Index Cond: tenant_id = ? and created_at < ?
   Filter: status = 'OPEN'
```

The index condition locates candidates; a filter is applied afterward. If millions of candidates are filtered to fifty rows, the index lacks a useful predicate. Repeated loops multiply node cost, especially in nested loops.

EXPLAIN ANALYZE executes the query. Use caution with writes and expensive production statements. Run in a safe transaction where appropriate, use representative data, and inspect buffer information rather than focusing only on elapsed time from a warm cache.

Sequential Scan Is Not Automatically Bad

A sequential scan is efficient for small tables or queries returning a large fraction of rows. Forcing an index can increase random page access. The goal is not "all queries use indexes" but predictable resource use within the service objective.

If an unexpectedly large production table is scanned, ask whether the query is missing a predicate, statistics are stale, an implicit cast prevents matching, a function wraps the indexed column, or the result genuinely needs most rows.

Sargable Predicates

A predicate is sargable when the engine can use an access structure effectively.

```
-- difficult for a normal index on created_at
where date(created_at) = :day

-- range can use created_at ordering
where created_at >= :start and created_at < :nextDay
```

Leading-wildcard search such as **LIKE** '%term', arithmetic on the indexed column, incompatible types, and implicit casts can block ordinary index use. Expression indexes can support a stable transformed predicate, but couple the index to that exact expression.

Covering and Included Columns

If all required columns are available from the index, the engine may avoid many table lookups through an index-only plan. Some databases support included non-key columns so projection can be covered without changing key order.

Covering is not free. Wider indexes consume memory, increase write amplification, and reduce fan-out. Visibility rules may still require table access. Cover only high-value query shapes proven by plans.

Partial and Functional Indexes

A partial index stores rows matching a predicate:

```
create index ix_orders_open_created
on orders(tenant_id, created_at desc)
where status = 'OPEN';
```

It is smaller and cheaper when queries consistently use the same condition. Queries whose predicates do not imply the partial condition cannot use it safely. Functional indexes support expressions such as normalized email, but the query must use a compatible expression and collation.

Joins and Join Algorithms

Nested loop joins are effective when the outer side is small and the inner lookup is indexed. Hash joins suit larger equality joins by building a hash table. Merge joins exploit sorted inputs. The optimizer selects from estimates and available memory.

```
Nested Loop
  outer: 10 orders
  inner: indexed customer lookup x 10

Bad case:
  outer estimate 10, actual 1,000,000
  inner lookup repeated 1,000,000 times
```

Index foreign-key columns used for joins and parent deletion checks where workload requires it. A declared foreign key does not universally create the child-side index automatically.

Sorting, Grouping, and Memory Spill

Sort and hash operations require working memory. When they exceed per-operation allowance, they spill to temporary disk and latency rises. Increasing memory globally is dangerous because many concurrent nodes can allocate it.

An index can provide order and avoid sorting when filters and ordering align. But fetching a huge result in index order may still be slower than scan plus sort. Reduce rows early, paginate, and aggregate at the correct layer.

Pagination

Offset pagination makes the database find and discard earlier rows. Large offsets grow expensive and concurrent inserts create duplicates or skips.

```
where tenant_id = :tenant
  and (created_at, id) < (:lastCreated, :lastId)
order by created_at desc, id desc
limit 50;
```

Keyset pagination seeks from the last ordered key. It needs deterministic ordering, a unique tie-breaker, a compatible index, and an opaque validated cursor. It trades arbitrary page numbers for stable scalable traversal.

Write Amplification and Redundant Indexes

Every insert, update of indexed keys, and delete maintains indexes and generates log or WAL. Too many indexes increase write latency, storage, cache pressure, vacuum work, replication volume, and migration time.

An index on (`tenant_id`) may be redundant when (`tenant_id, created_at`) serves the same important lookups, but differences in width and workload can still matter. Use index-usage evidence and dependency checks before removal. Constraints may rely on an index.

Index Build and Deployment

Creating an index on a large live table can scan the table, consume I/O, generate replication lag, and block or wait depending on engine and build mode. Online or concurrent builds reduce blocking but take longer and have failure states to monitor.

Roll out in stages: create safely, verify validity and replica health, deploy query changes, confirm plan and latency, then remove old structures later. Schema migration timeout and application startup are poor places for an uncontrolled multi-hour build.

JPA and ORM Query Visibility

JPQL, Criteria, and derived repository methods eventually become SQL. Inspect generated SQL with bound values in safe environments. Concise object navigation can trigger joins, N+1 queries, wide selects, or unnecessary counts.

Do not add an index for a query that should not exist. Fetching every entity and filtering in Java wastes database filtering and network capacity. Conversely, forcing complex business logic into one unreadable SQL statement may harm maintainability. Profile the boundary.

Partitioning Is Not an Index Replacement

Table partitioning divides data into physical children by range, list, or hash. It helps lifecycle operations, very large tables, parallel maintenance, and queries whose predicates let the optimizer prune most partitions.

```
orders partitioned by created_month
orders_2026_06
orders_2026_07
orders_2026_08
```

A query lacking the partition key may scan every partition, sometimes with more planning overhead than one table. Each partition still needs appropriate indexes. Choose a key aligned with retention and common access, and test uniqueness semantics because some databases require partition keys in globally unique constraints.

Too many tiny partitions increase catalog, planning, and maintenance cost. Partitioning by tenant can create thousands of uneven objects and operational complexity. Use it after ordinary schema, indexing, and archiving approaches are understood.

Plan Stability and Parameter Sensitivity

Prepared statements may reuse a generic plan or compile a plan from observed parameters, depending on database and driver behavior. When data is skewed, one plan does not suit every value: an index lookup is excellent for a small tenant while a scan may be cheaper for the dominant tenant.

Do not solve this immediately with hints. Confirm the selected plan and parameter class, improve statistics, consider extended statistics for correlated columns, or separate genuinely different query shapes. Hints and forced plans can freeze assumptions that become wrong as data grows.

Upgrades can change optimizer behavior, cost models, parallelism, and cardinality estimates. Capture representative plans and latency before and after database, driver, or schema upgrades. Treat plan changes as evidence to review, not automatically as regressions.

Query Shape, Round Trips, and Batching

Database performance is end-to-end. One 30 ms set-based query can be far cheaper than 1,000 sub-millisecond lookups because round trips, connection occupancy, parsing, and repeated index traversal accumulate.

Batch writes with prepared statements where semantics permit, but account for generated keys, optimistic versions, constraint failure attribution, and transaction size. Very large batches retain locks and generate recovery work; use bounded chunks with restartable progress.

Avoid selecting unused wide columns, large JSON, or LOB values. Projection reduces network and heap cost and may enable index-only access. Count queries deserve their own plan; automatically calculating an exact total for every paginated response can dominate the page query.

Capacity and Connection Pools

A fast query can still wait behind a saturated connection pool. Increasing pool size beyond database CPU and I/O capacity increases concurrency and often worsens queueing. Coordinate application replicas, per-instance connections, administrative headroom, and database limits.

Measure acquisition time separately from execution. Apply deadlines and cancellation so abandoned requests do not keep consuming database work. Cancellation is driver and engine dependent, so verify that a timed-out Java future actually stops or bounds the SQL.

Slow Query Diagnosis

Start with a normalized query fingerprint and latency distribution. Capture plan, actual parameters or representative classes, data volume, row estimates, buffers, lock waits, and concurrency. A query can be slow because it waited on a lock even when its plan is good.

Check recent deployments, statistics, schema changes, cache state, table growth, skew, connection saturation, temporary I/O, and replica role. Preserve evidence before restarting or blindly adding an index.

Testing Performance

Use production-like data distribution, not only row count. Include large tenants, common and rare statuses, null distribution, and realistic correlations. Warm and cold cache behavior differ. Test concurrent load because an excellent single-query plan may saturate I/O at production arrival rates.

Regression tests can assert query count and broad latency budgets, but exact plan text is version-sensitive. Prefer assertions on critical access behavior and compare plans during controlled upgrades.

Common Senior-Level Traps

Traps include indexing every WHERE column independently, insisting sequential scan is failure, choosing composite order from selectivity alone, ignoring write cost, using offset at extreme depth, and reading estimated cost as milliseconds.

Another is tuning with one parameter value. Parameter-sensitive workloads require representative classes, query variants, or engine-specific plan controls after evidence.

A Strong Interview Answer

I design indexes from complete query shapes: equality and range predicates, join keys, ordering, limit, and projected columns. I inspect actual plans and compare estimated to actual rows, loops, buffers, sort spills, and lock time. Sequential scans are valid when much of a table is needed. I keep predicates sargable, use composite, partial, expression, or covering indexes only for measured workloads, and account for write amplification and build risk. For deep ordered traversal I prefer keyset pagination with a deterministic tie-breaker. Performance tests use production-like distribution and concurrency, and I remove redundant indexes only after usage and constraint analysis.

Interview-Ready Summary

Indexes are workload-specific access structures, not universal accelerators. Optimizer decisions depend on row estimates, distribution, cache and I/O cost. Execution plans show how data is actually found, joined, filtered, sorted, and repeated.

Good performance comes from reducing work, aligning indexes with query order, maintaining statistics, bounding results, and considering writes and deployment. Diagnose waits and capacity alongside SQL plan quality.

Revision Notes

- B-tree indexes support equality, ranges, and ordered prefix access.
- Index usefulness depends on result fraction and page cost.
- Design composite indexes from the entire query shape.
- Equality, range, ordering, and tie-breakers influence key order.
- Statistics and data skew drive optimizer estimates.
- Compare estimated rows with actual rows and loops.
- Read execution plans from data-producing nodes upward.
- Index conditions locate rows; filters discard candidates afterward.
- Sequential scans are appropriate for small or broad-result queries.
- Keep predicates sargable or use deliberate expression indexes.
- Covering indexes reduce table access but increase write and storage cost.
- Partial indexes serve queries implying their predicate.
- Join algorithms depend on cardinality, ordering, and memory.
- Sort and hash spills create temporary I/O.
- Keyset pagination avoids large offset traversal.
- Every index adds write amplification and operational cost.
- Build large indexes with deployment and replica impact controls.
- ORM abstractions do not remove generated SQL and query-plan behavior.
- Diagnose lock waiting separately from execution time.
- Test realistic distribution, skew, parameters, and concurrency.